

プログラムの設計方法 第二版(日本語訳・抜粋)

Felleisen et al. 訳

プログラムの設計方法 第二版

How to Design Programs, Second Edition

著者: Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

© 2014 (更新版 2026) MIT Press

この資料は著作権保護されており、Creative Commons [CC BY-NC-ND](https://creativecommons.org/licenses/by-nc-nd/4.0/) ライセンスの下で提供されています。

目次

前付け

- [序文 \(Preface\)](#)
 - 体系的なプログラム設計
 - DrRacket とティーチング言語
 - 移行可能なスキル
 - この本とその各部
 - 違い点
- [プロローグ: プログラムの仕方 \(Prologue: How to Program\)](#)
 - 算術と算術
 - 入力と出力
 - 計算のさまざまな方法
 - 1つのプログラム、多くの定義
 - もう1つの定義
 - あなたはもうプログラマだ
 - 違う!

本編

I 固定サイズのデータ (Fixed-Size Data)

- [1 算術](#)
 - 1.1 数の算術
 - 1.2 文字列の算術
 - 1.3 混ぜ合わせ

- 1.4 画像の算術
- 1.5 ブール値の算術
- 1.6 ブール値と混ぜ合わせ
- 1.7 述語: 自分のデータを知れ
- 2 関数とプログラム
 - 2.1 関数
 - 2.2 計算
 - 2.3 関数の合成
 - 2.4 グローバル定数
 - 2.5 プログラム
- 3 プログラムの設計方法 (How to Design Programs)
 - 3.1 関数の設計
 - 3.2 指の運動: 関数
 - 3.3 ドメイン知識
 - 3.4 関数からプログラムへ
 - 3.5 テストについて
 - 3.6 World プログラムの設計
 - 3.7 バーチャルペットの世界
- 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)
 - 4.1 条件式を使ったプログラミング
 - 4.2 条件付き計算
 - 4.3 列挙
 - 4.4 区間
 - 4.5 項目化
 - 4.6 項目化を使った設計
 - 4.7 有限状態の世界
- 5 構造の追加 (Adding Structure)
 - 5.1 位置から posn 構造へ
 - 5.2 posn を使った計算
 - 5.3 posn を使ったプログラミング
 - 5.4 構造型定義
 - 5.5 構造を使った計算
 - 5.6 構造を使ったプログラミング
 - 5.7 データの宇宙
 - 5.8 構造を使った設計
 - 5.9 World の中の構造
 - 5.10 グラフィカルエディタ
 - 5.11 さらに多くのバーチャルペット
- 6 項目化と構造 (Itemizations and Structures)
 - 6.1 再び、項目化を使った設計
 - 6.2 World を混ぜる
 - 6.3 入力エラー
 - 6.4 World の検査
 - 6.5 等価述語

- ・ [7 まとめ](#)

Intermezzo 1: Beginning Student Language

[BSL の語彙、文法、意味、エラーなど](#)

II 任意サイズのデータ (Arbitrarily Large Data)

- ・ [8 リスト \(Lists\)](#)
- ・ [9 自己参照データ定義を使った設計](#)
- ・ [10 リストの続き](#)
- ・ [11 合成による設計 \(Design by Composition\)](#)
- ・ [12 プロジェクト: リスト](#)
 - 実世界データ: 辞書、iTunes
 - 単語ゲーム、ワーム、単純テトリス、完全宇宙戦争、有限状態機械
- ・ [13 まとめ](#)

Intermezzo 2: Quote, Unquote

III 抽象化 (Abstraction)

- ・ [14 どこにでも類似点がある](#)
- ・ [15 抽象化の設計](#)
- ・ [16 抽象化の使用](#)
- ・ [17 無名関数 \(Nameless Functions / lambda\)](#)
- ・ [18 まとめ](#)

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ (Intertwined Data)

- ・ [19 S 式の詩 \(The Poetry of S-expressions\)](#)
- ・ [20 反復的洗練 \(Iterative Refinement\)](#)
- ・ [21 インタプリタの洗練](#)
- ・ [22 プロジェクト: XML の商業](#)
- ・ [23 同時処理](#)
- ・ [24 まとめ](#)

Intermezzo 4: The Nature of Numbers

V 生成的再帰 (Generative Recursion)

- ・ [25 非標準の再帰](#)
- ・ [26 アルゴリズムの設計](#)
- ・ [27 バリエーション](#)
- ・ [28 数学的な例](#)
- ・ [29 バックトラッキングするアルゴリズム](#)
- ・ [30 まとめ](#)

Intermezzo 5: The Cost of Computation

VI 蓄積子 (Accumulators)

- ・ [31 知識の喪失](#)

- [32 蓄積子スタイル関数の設計](#)
- [33 蓄積のさらなる使用](#)
- [34 まとめ](#)

エピローグ: 先へ進む (Epilogue: Moving On)

注意: 本翻訳は学習・個人利用のためのものです。サンプルコードは原文のまま保持しています。

原典: <https://htdp.org/2026-5-28/Book/index.html> # 序文

Preface

	序文
I	プロローグ: プログラムの仕方
	固定サイズのデータ
II	Intermezzo 1: Beginning Student Language
	任意サイズのデータ
III	Intermezzo 2: Quote, Unquote
	抽象化
IV	Intermezzo 3: Scope and Abstraction
	絡み合ったデータ
V	Intermezzo 4: The Nature of Numbers
	生成的再帰
VI	Intermezzo 5: The Cost of Computation
	蓄積子
	エピローグ: 先へ進む

多くの職業で何らかのプログラミングが必要です。会計士はスプレッドシートをプログラムし、ミュージシャンはシンセサイザーをプログラムし、著者はワードプロセッサをプログラムし、Webデザイナーはスタイルシートをプログラムします。私たちが初版のこれらの言葉を書いたとき(1995–2000)、読者はそれらを未来的だと考えたかもしれません。今や、プログラミングは必須のスキルとなり、雇用見通しを高めるという目的で、数多くの書籍、オンラインコース、K-12 カリキュラムがこのニーズに応えています。

典型的なプログラミングのコースでは、「動くまでいじくり回す」アプローチを教えます。それが動くと、学生は「動いた!」と叫んで次に進みます。悲しいことに、このフレーズはコンピューティングにおける最短の嘘でもあり、多くの人々の人生から何時間も奪ってきました。対照的に、本書は良いプログラミングの習慣に焦点を当て、プロフェッショナルおよび職業プログラマの両方を対象としています。

「良いプログラミング」とは、ソフトウェア作成へのアプローチで、最初から、すべての段階で、すべてのステップで体系的な思考、計画、理解に依拠するものを意味します。この点を強調するために、私たちは「体系的なプログラム設計」と「体系的に設計されたプログラム」について語りまします。後者は、望まれる機能の根拠を明確に述べます。良いプログラミングは達成感の美的感覚も満たします。良いプログラムのエレガンスは、時の試練に耐えた詩や過去の時代の白黒写真に匹敵します。要するに、プログラミングと良いプログラミングの違いは、ダイナーのクレヨン画と

美術館の油絵のようなものです。

いいえ、この本は誰をもマスターペインターには変えません。しかし、私たちがこの版を書くのに 15 年を費やしたのは、以下を信じているからに他なりません。

誰もがプログラムを設計できる

そして

誰もが創造的設計から得られる満足感を体験できる

実際、私たちはさらに進んで主張します。

プログラム設計——しかしプログラミングではない——は、リベラルアーツ教育において数学や言語スキルと同じ役割を果たすべきである。

設計を学ぶ学生が二度とプログラムに触れなくても、普遍的に有用な問題解決スキルを身につけ、深く創造的な活動を経験し、新しい形式の美を理解することを学ぶでしょう。この序文の残りでは、「体系的設計」とは何を意味するのか、誰がどのように恩恵を受けるのか、そしてそれをどのように教えるのかを詳しく説明します。

体系的なプログラム設計

プログラムは人々（ユーザーと呼ばれる）と他のプログラム（サーバーとクライアントコンポーネントの場合）と相互作用します。したがって、合理的に完全なプログラムは多くの構成要素から成ります。一部は入力扱い、一部は出力を作成し、一部はそれらの間のギャップを橋渡しします。私たちは関数を基本的な構成要素として選ぶ理由は、誰もが前代数で関数に遭遇するからであり、最も単純なプログラムがまさにそのような関数だからです。鍵は、どの関数が必要か、それらをどのように接続するか、そして基本的な材料からそれらをどのように構築するかを発見することです。

この文脈で、「体系的なプログラム設計」とは、2つの概念の混合を指します。**設計レシピ**と**反復的洗練**です。私たちは Michael Jackson の COBOL プログラム作成法、Daniel Friedman との再帰に関する会話、Robert Harper との型理論、Daniel Jackson とのソフトウェア設計から着想を得ました。設計レシピは著者らの創作であり、ここでは後者の使用を可能にします。

問題分析からデータ定義へ 表現しなければならない情報と、それが選択したプログラミング言語でどのように表現されるかを特定せよ。データ定義を定式化し、例で説明せよ。署名、目的文、ヘッダ 望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。その関数が何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に沿ったスタブを定義せよ。関数の例 関数の目的を説明する例に取り組み。関数のテンプレート データ定義を関数のアウトラインに翻訳せよ。関数定義 関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。テスト 例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。これにより誤りが発見される。テストはまた、必要が生じたときに他の人が定義を読み理解するのを助ける——そしてそれは深刻なプログラムでは必ず生じる。

図 1: 関数設計レシピの基本ステップ

設計レシピは完全なプログラムと個々の関数の両方に適用されます。本書は完全なプログラムのための 2 つのレシピだけを扱います。GUI を持つプログラム用とバッチプログラム用です。—

方、関数用の設計レシピにはさまざまなバリエーションがあります。数値のような原子的なデータ形式用、さまざまな種類のデータの列挙用、固定された方法で他のデータを複合するデータ用、有限だが任意に大きいデータ用、など。

関数レベルの設計レシピは共通の設計プロセスを共有します。図 1 はその 6 つの必須ステップを示しています。各ステップのタイトルは期待される成果を指定し、「コマンド」は主要な活動を示唆します。例はほぼすべての段階で中心的な役割を果たします。

各ステップには鋭い質問が付随しており、本書の 6 つのパートを通じて導入されます。特定のステップ—例えば関数の例の作成やテンプレート—では、質問はデータ定義に訴えかけるかもしれませんが。答えはほぼ自動的に中間成果物を作成します。

このアプローチの新しさは、初心者レベルのプログラムのための中間成果物の作成にあります。初心者が詰まったとき、専門家や講師は既存の中間成果物を検査できます。検査は設計プロセスの一般的な質問を使う可能性が高く、したがって初心者に自分で修正させることになります。そしてこの自己強化プロセスこそが、プログラミングとプログラム設計の重要な違いです。

反復的洗練は、問題が複雑で多面的であるという問題に対処します。一度にすべてを正しくすることはほぼ不可能です。代わりに、コンピュータ科学者はこの設計問題に取り組むために物理学から反復的洗練を借用します。本質的に、反復的洗練は、最初にすべての非本質的な詳細を取り除き、残された中核問題の解決策を見つけることを推奨します。洗練ステップは、これらの省略された詳細の 1 つを追加し、既存の解決策を可能な限り使用して拡張された問題を再解決します。これらの洗練ステップの繰り返し（反復とも呼ばれる）は、最終的に完全な解決策につながります。

この意味で、プログラマはミニ科学者です。科学者は世界の理想化されたバージョンのための近似モデルを作成し、それについての予測を行います。モデルの予測が当たっている限り、すべてはうまくいきます。予測された出来事が実際のもものと異なる場合、科学者はその不一致を減らすためにモデルを改訂します。同様に、プログラマにタスクが与えられたとき、彼らは最初の設計を作成し、それをコードにし、実際のユーザーで評価し、プログラムの振る舞いが望まれる製品に密接に一致するまで設計を反復的に洗練します。

本書は 2 つの異なる方法で反復的洗練を導入します。洗練による設計がプログラムの設計が複雑になると有用になるため、本書は問題がある程度の難易度を獲得した第 4 部でこの技法を明示的に導入します。さらに、私たちは第 1 部から第 3 部を通じて同じ問題のますます複雑なバリエーションを述べるために反復的洗練を使用します。つまり、中核問題を選び、1 つの章でそれを扱い、その後、後続の章で新たに導入された概念に合致する詳細で類似の問題を提示します。

DrRacket とティーチング言語

プログラムを設計することを学ぶには、繰り返しのハンズオン練習が必要です。ピアノを弾かずにピアノ奏者になれないのと同じように、実際のプログラムを作成して正しく動作させることなしにプログラム設計者にはなりません。したがって、本書には最小限のソフトウェアサポートが付属しています。プログラムを書き留めるための言語と、プログラムをワード文書のように編集し、プログラムを実行できるプログラム開発環境です。

多くの人が「コードの書き方を学びたい」と言い、次にどのプログラミング言語を学ぶべきかを尋ねます。いくつかのプログラミング言語が得ている注目を考えると、この質問は驚くべきことではありません。しかし、それは全く不適切でもあります。初心者を対象としないコースでは、既製の言語を設計レシピとともに使用できるかもしれません。現在流行のプログラミング言語でプロ

プログラミングを学ぶことは、学生を最終的な失敗に導くことがよくあります。この世界での流行は非常に短命です。「X で素早くプログラミング」本やコースは、次の流行の言語に移行できる原則を教えることに失敗することが多いです。さらに悪いことに、言語自体が、解決策を表現するレベルでもプログラミングミスに対処するレベルでも、移行可能なスキルの習得から気を逸らします。

対照的に、プログラムを設計することを学ぶことは、主に原則の研究と移行可能なスキルの習得についてです。理想的なプログラミング言語はこれら2つの目標をサポートしなければなりませんが、既製の産業用言語はそうしません。重要な問題は、初心者が言語の多くを知る前に間違いを犯すことです。しかしプログラミング言語は、プログラマがすでに言語全体を知っているかのように常にこれらのエラーを診断します。その結果、診断レポートは初心者をしばしば困惑させます。

私たちの解決策は、私たち自身の仕立てられたティーチング言語から始めることです。「Beginning Student Language」またはBSLと呼ばれます。この言語は本質的に、学生が前代数のコースで習得する「外国語」です。関数定義、関数適用、条件式のための記法を含みます。また、式を入れ子にすることもできます。この言語は非常に小さいので、言語全体の観点からのエラー診断でも、前代数しか持たない読者にとってまだアクセス可能です。

構造的設計原則を習得した学生は、次に「Intermediate Student Language」と他の高度な方言（総称して*SL）に進むことができます。本書はこれらの方言を使用して抽象化と一般再帰の設計原則を教えます。私たちは、このような一連のティーチング言語を使用することが、幅広い専門プログラミング言語（JavaScript、Python、Ruby、Java など）でプログラムを作成するための優れた準備を読者に提供すると固く信じています。

注: ティーチング言語は Racket で実装されています。Racket はプログラミング言語を構築するためのプログラミング言語として私たちが構築したものです。Racket はラボから現実世界に逃れ出しました。それはプログラミング言語です。

（続き: DrRacket の使用、ティーチング言語の選択など詳細）

移行可能なスキル

（中略: 設計レシピから得られる普遍的な問題解決スキル、創造的活動としてのプログラミングの価値について）

この本とその各部

本書は6つの主要なパートと5つの Intermezzo に分かれています。各パートは設計レシピの特定の側面に焦点を当て、徐々に複雑さを増していきます。

- Part I: 固定サイズのデータ — 基本的な算術から設計レシピの導入まで。
- Part II: 任意サイズのデータ — リストと自己参照データ定義。
- Part III: 抽象化 — 類似性の発見と高階関数。
- Part IV: 絡み合ったデータ — 複雑なデータ定義の相互作用。
- Part V: 生成的再帰 — 構造的再帰を超えたアルゴリズム。
- Part VI: 蓄積子 — 効率的な再帰のための追加の知識の保持。

各 Intermezzo では、言語の形式的な側面（文法、意味、計算モデル）を扱います。

違い点

(初版との違いの説明:より明確な設計レシピ、プロジェクトの強化など)

謝辞 (Acknowledgments)

…(詳細は原典を参照。多くの貢献者への感謝)

本翻訳は原典の内容を忠実に日本語化したものです。サンプルコードは原文を維持しています。

原典 URL: https://htdp.org/2026-5-28/Book/part_preface.html # プロローグ: プログラムの仕方

Prologue: How to Program

(目次リンクは省略)

あなたが小さな子供だったとき、親はあなたに数を数え、指で簡単な計算をすることを教えました。「 $1 + 1$ は 2」;「 $1 + 2$ は 3」;など。そして「 $3 + 2$ は？」と尋ね、あなたは片手の指を数えました。彼らはプログラムし、あなたは計算しました。そしてある意味で、それが本当にプログラミングとコンピューティングのすべてです。

今度は役割を交代する時です。DrRacket を起動しましょう。そうすると DrRacket のウィンドウが表示されます。「Language」メニューから「Choose language」を選び、「How to Design Programs」の「Teaching Languages」から「Beginning Student」(BSL)を選択し、OK をクリックして DrRacket を設定します。

これで準備完了です。あなたがプログラムし、DrRacket ソフトウェアが子供の役割を果たします。

最も単純な計算から始めましょう。DrRacket の上部に次のように入力します:

(+ 1 1)

RUN をクリックすると、下部に 2 が表示されます。

(図: DrRacket の画面)

それほど単純なのがプログラミングです。DrRacket を子供のように質問し、DrRacket があなたのために計算します。一度に複数のリクエストを処理させることもできます:

(+ 2 2)

(* 3 3)

(- 4 2)

(/ 6 2)

RUN をクリックすると、期待される結果 4 9 2 3 が表示されます。

少しペースを落として用語を導入しましょう。

DrRacket の上半分は「定義領域 (definitions area)」と呼ばれます。この領域でプログラムを作成します。これを編集と呼びます。定義領域に単語を追加したり変更したりすると、左上に SAVE ボタンが表示されます。初めて SAVE をクリックすると、DrRacket はファイルを保存するた

めの名前を尋ねます。一度ファイルに関連付けられると、SAVE をクリックすると定義領域の内容が安全にファイルに保存されます。

プログラムは式 (expressions) で構成されます。数学で式に出会ったことがあるでしょう。今のところ、式は単なる数値か、左括弧「(」で始まり対応する右括弧「)」で終わるものです——DrRacket はその括弧の間の領域を強調表示します。

RUN をクリックすると、DrRacket は定義領域の式を評価し、その結果を対話領域 (interactions area) に表示します。その後、DrRacket はプロンプト (>) であなたのコマンドを待ちます。プロンプトが表示されたら、追加の式を入力でき、DrRacket は定義領域と同じようにそれを評価します。

このようなプログラミングとコンピューティングを DrRacket で簡単に探求できます。

算術と算術

プログラミングが数と算術だけだったら、数学と同じくらい退屈でしょう。幸い、プログラミングには数値以外にも多くのものがあります。テキスト、真偽値、画像などです。

BSL で知っておくべき最初のことは、テキストは二重引用符 (") で囲まれた任意のキーボード文字のシーケンスであるということです。これを文字列 (string) と呼びます。したがって、

```
"hello world"
```

は完全に正しい文字列であり、DrRacket がこの文字列を評価すると、対話領域で数値と同じようにそのままエコーバックします。

BSL には数の算術に加えて、文字列の算術もあります。以下に 2 つの例を示します：

```
(string-append "hello" "world")  
; "helloworld"
```

```
(string-append "hello " "world")  
; "hello world"
```

+ と同様に、string-append は演算子です。2 番目の文字列を最初の文字列の末尾に追加して新しい文字列を作ります。最初の例が示すように、2 つの文字列の間に何も挿入せずに文字通り結合します。空白、カンマ、何もありません。

(以下、画像の算術、ブール値、条件式、関数の定義などプロローグの続きが展開されます。)

多くの方法で計算する

(条件式の導入例)

1 つのプログラム、多くの定義

(define を使った定数と関数の導入)

あなたはもうプログラマだ

ここまで来たら、あなたはすでに小さなプログラムを書けるプログラマです。以降の章で設計レ

レシピを学び、体系的にプログラムを構築していきます。

注意: サンプルコードは原文のままです。

原典: https://htdp.org/2026-5-28/Book/part_prologue.html # 3 プログラムの設計方法

3.1 関数の設計

すべてのプログラミング言語には、データの言語とデータに対する演算の言語が備わっています。最初の言語は常に何らかの形の原子データを提供します。現実世界のさまざまな情報をデータとして表現するために、プログラマは基本データを合成し、そうした合成を記述することを学ばなければなりません。同様に、2 番目の言語は原子データに対する基本演算を提供します。プログラマの仕事は、これらの演算を合成して望まれる計算を行うプログラムを作ることです。

私たちはプログラミング言語のこれら 2 つの部分の組み合わせに「算術」を使います。これは小学校で学んだことを一般化したものだからです。

本書の第 I 部は、プロローグで使ったプログラミング言語 BSL の算術を紹介します。算術から、数学で関数として知っているかもしれない最初の単純なプログラムへは短いステップです。しかしすぐに、プログラムを書くプロセスは混乱して見え、思考を整理する方法を切望するようになるでしょう。私たちは「思考を整理すること」を **設計** と同一視し、本書のこの最初の部分で体系的なプログラム設計の方法を紹介します。

関数の設計レシピ

プログラムを書くとき、特に初心者のうちは「何から始めればいいのかわからない」という状態に陥りがちです。本書が提供する解決策が **設計レシピ (design recipe)** です。

設計レシピは、関数を設計するための一連の体系的なステップです。以下の図がその基本ステップを示しています。

図1: 関数設計レシピの基本ステップ

1. **問題分析からデータ定義へ**
表現しなければならない情報を特定し、それが選択したプログラミング言語でどのように表現されるかを明らかにせよ。データ定義を定式化し、例で説明せよ。
2. **署名、目的文、ヘッダ**
望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。関数が何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に合致するスタブ (仮の実装) を定義せよ。
3. **関数の例**
関数の目的を具体的に示す例に取り組み。
4. **関数のテンプレート**
データ定義を関数のアウトライン (骨組み) に翻訳せよ。
5. **関数定義**
関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。

6. テスト

例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。

これらのステップは、初心者が迷ったときに「今どのステップにいるか」を明確にし、中間成果物を検査することで修正を導くことができます。

実際の例: 距離を計算する関数

Cartesian 平面上の点 (x, y) から原点 $(0, 0)$ までの距離を計算する関数を考えます。

ステップ 1: データ定義

点は 2 つの数値 (x, y) で表される。Number 型のデータ。

ステップ 2: 署名と目的文

```
; dist : Number Number -> Number
; 点(x, y) から原点までの距離を計算する
(define (dist x y) 0)
```

ステップ 3: 例の作成

```
(check-expect (dist 3 4) 5)
(check-expect (dist 0 0) 0)
(check-expect (dist 12 5) 13)
```

ステップ 4: テンプレート

(ここではシンプルなので距離公式を直接使うが、一般にデータ定義から骨組みを作る)

ステップ 5: 定義の完成

```
(define (dist x y)
  (sqrt (+ (sqr x) (sqr y))))
```

ステップ 6: テスト実行

DrRacket で RUN し、すべての check-expect が緑になることを確認。

(注: 上記のコードはサンプルとして原文の BSL 記法を維持しています。)

指の運動: 関数の練習問題

本書では、読者が実際に手を動かして例を書くことを強く推奨します。これを「指の運動 (finger exercises)」と呼びます。

例: - 華氏から摂氏への変換関数 - 文字列の結合と長さ計算 - 画像の操作 (後述)

(詳細な練習問題は各章に多数収録)

3.2 指の運動: 関数

(本文では多数の小さな関数設計練習が用意されています。すべてコード例は原文保持。)

3.3 ドメイン知識

問題を解決するには、その問題領域に関する知識(ドメイン知識)が必要です。数学の公式、物理法則、業務ルールなど。

設計レシピの最初に「問題を理解する」ステップがあるのはこのためです。

3.4 関数からプログラムへ

単一の関数から、複数の関数を組み合わせた完全なプログラムへ移行する方法を学びます。定数定義や補助関数の使用。

3.5 テストについて

良いテストとは: - 普通のケース - 境界値 - エラーケース

`check-expect` を使って自動化されたテストを書く。

3.6 World プログラムの設計

後の章で詳述されるが、グラフィカルでインタラクティブなプログラム(「世界」)のための設計レシピの導入。

3.7 バーチャルペットの世界

簡単なアニメーション付きペットシミュレータのプロジェクト例。

注意: 本章のすべての Racket コード例・式は原文のままです。翻訳は説明テキストのみ。

原典: https://htdp.org/2026-5-28/Book/part_one.html (Chapter 3)